

Полевая архитектура роя (Field-Based Architecture)

Данная работа представляет практическую и реализуемую архитектуру для распределённых рой-вычислительных систем. Предложенная модель рассматривает распределённую сеть как динамическое поле, где задачи и ресурсы формируют взаимодействующие потенциалы. Вместо полагания на центральные планировщики или глобальную координацию, система выполняет балансировку нагрузки через локальные взаимодействия между узлами.

Подход объединяет три слоя:

1. **Полевая эффективность на основе диффузии**
2. **Градиентно-управляемое поведение агентов**
3. **Динамика потока задач–ресурсов**

Результирующая структура обеспечивает децентрализованное планирование, адаптивную балансировку нагрузки и масштабируемую координацию через сотни или тысячи узлов с использованием только локальных коммуникаций.

Модель математически обоснована графовой динамикой диффузии и реакционно-диффузионными системами и реализуема на существующей распределённой инфраструктуре, такой как кластеры edge, рой-системы робототехники и ячеистые сети.

1. Введение

Распределённые системы обычно полагаются на явные механизмы планирования, централизованное оркестрирование или сложные протоколы координации. Хотя эти подходы хорошо работают в контролируемых средах, они становятся сложными для масштабирования или поддержки в высоко динамичных сетях.

Рой-системы предлагают альтернативную парадигму, где глобальная организация возникает из локальных взаимодействий.

Однако большинство рой-алгоритмов (таких как particle swarm optimization или ant colony optimization) предназначены для задач оптимизации, а не для реальной инфраструктуры.

Данная работа предлагает практическую рой-вычислительную модель, где вычисления ведут себя подобно физическим потоковым процессам.

Вместо центрального назначения задач узлы обмениваются простыми переменными состояниями. Из этих переменных глобальное поведение планирования возникает автоматически.

Основная идея: задачи движутся через сеть в соответствии с градиентами между плотностью задач и доступностью ресурсов.

2. Модель сети

Рассмотрим распределённую систему, представленную как граф:

$$G = (V, E)$$

где:

- V = узлы
- E = коммуникационные связи

Каждый узел общается только со своими соседями.

Соседей узла i обозначается как $N(i)$.

Типичные развертывания включают:

- кластеры edge
- ячеистые сети
- распределённые рой-вычисление
- сенсорные рой -сети

3. Представление состояния узла

Каждый узел i поддерживает небольшой вектор состояния.

Состояние узла:

- T_i — число или плотность задач
- R_i — доступные ресурсы
- E_i — оценка эффективности
- Φ_i — потенциал нагрузки
- **neighbors** — подключённые узлы

Ресурсы могут представлять время CPU, ёмкость памяти, энергию или пропускную способность.

4. Полевая эффективность

Сеть поддерживает распределённое скалярное поле, представляющее локальную эффективность.

Эффективность оценивается как:

$$E_i = \alpha R_i + \beta C_i + \gamma B_i$$

где:

- R_i — доступность ресурсов
- C_i — ёмкость вычислений
- B_i — пропускная способность сети

$$\alpha + \beta + \gamma = 1$$

Это значение представляет, насколько привлекателен узел для выполнения задач.

5. Диффузия поля

Информация об эффективности распространяется в сети через обмен между соседями.

Дискретное диффузионное уравнение:

$$E_i(t+1) = E_i(t) + D \sum_{j \in N(i)} (E_j - E_i) + S_i - \lambda E_i$$

Где:

- D — коэффициент диффузии
- S_i — локальный член источника
- λ — фактор затухания

Этот процесс позволяет информации об эффективности распространяться через сеть без централизованной координации.

6. Агентное движение задач

Задачи или мобильные процессы могут двигаться между узлами.

Движение следует градиенту поля эффективности.

Концептуально: $\text{velocity} \propto \text{gradient}(E)$

Задачи таким образом мигрируют к узлам, где выполнение наиболее эффективно.

Случайный шум может быть добавлен для разрешения исследования.

7. Потенциал задач–ресурсов

Для моделирования баланса нагрузки напрямую мы вводим потенциальное поле:

$$\Phi_i = T_i - R_i$$

Где:

- T_i — плотность задач
- R_i — доступность ресурсов

Интерпретация:

- $\Phi_i > 0 \rightarrow$ загруженный узел
 - $\Phi_i < 0 \rightarrow$ незагруженный узел
 - $\Phi_i \approx 0 \rightarrow$ сбалансированный узел
-

8. Уравнение потока задач

Задачи движутся через сеть в соответствии с разницей потенциалов.

Поток между узлами i и j :

$$J_{ij} = -k (\Phi_j - \Phi_i)$$

Где k — коэффициент мобильности.

Задачи естественно движутся к узлам с большей доступностью ресурсов.

9. Сохранение задач

Плотность задач эволюционирует согласно:

$$T_i(t+1) = T_i + \sum_j J_{ji} - \sum_j J_{ij} + G_i$$

Где G_i представляет создание новых задач.

Это уравнение сохраняет общее число задач в сети.

10. Динамика ресурсов

Ресурсы потребляются задачами и пополняются со временем.

Упрощённая модель:

$$R_i(t+1) = R_i - C(T_i, R_i) + S_i$$

Где:

- C представляет потребление ресурсов задачами
 - S_i представляет новые доступные ресурсы.
-

11. Энергетическая интерпретация системы

Глобальный дисбаланс системы может быть представлен функционалом энергии:

$$H = \sum_i (T_i - R_i)^2$$

Динамика роя стремится уменьшить эту энергию.

Таким образом, система естественно эволюционирует к сбалансированному распределению нагрузки.

12. Устойчивость поля

Процесс диффузии может быть записан в матричной форме используя графический лапласиан L .

Динамика поля:

$$dE/dt = -DL E - \lambda E + S$$

Если $D > 0$ и $\lambda > 0$, все возмущения затухают со временем.

Это обеспечивает устойчивое поведение поля эффективности.

13. Масштабируемость

Алгоритм требует только соседних коммуникаций.

Каждый узел выполняет $O(k)$ работы на обновление, где k — степень узла.

Это позволяет системе масштабироваться до больших сетей.

На практике архитектура может работать эффективно с: • сотнями узлов • тысячами узлов при умеренных нагрузках коммуникации.

14. Алгоритм обновления узла

Каждый узел выполняет следующий цикл.

Алгоритм:

1. Измеряет локальные ресурсы
2. Вычисляет потенциал Φ_i

3. Обменивает Φ_i с соседями
4. Вычисляет потоки J_{ij}
5. Обновляет счётчики задач
6. Выполняет задачи

Этот процесс может работать асинхронно через сеть.

15. Архитектура реализации

Практическая система может быть реализована с помощью четырёх слоёв.

Swarm Kernel (ядро роя)

Поддерживает поля и выполняет обновления диффузии.

Node Runtime (время выполнения узла)

Отслеживает локальное состояние и ресурсы.

Task Scheduler (планировщик задач)

Обработка миграции задач между узлами.

Communication Layer (коммуникационный уровень)

Обменивается состоянием с соседними узлами.

16. Практическое развертывание

Возможные реализации включают:

- распределённые кластеры edge
- робототехнические узлы вычислений
- децентрализованная облачная инфраструктура
- ячеистые сенсорные сети

Типичное количество узлов варьируется от десятков до тысяч.

Узлы общаются с помощью лёгких периодических сообщений, содержащих:

- Φ_i — потенциал нагрузки
 - R_i — доступные ресурсы
 - T_i — плотность задач
-

17. Сравнение с традиционным планированием

Традиционные распределённые системы используют:

- центральные планировщики
- выборы лидеров
- сложную координацию

Модель рой-поля заменяет эти механизмы непрерывным локальным балансирующим.

Преимущества включают:

- устойчивость к сбоям
 - отсутствие единой точки отказа
 - адаптивная балансировка нагрузки
 - простая логика узла
-

18. Ограничения

Модель предполагает:

- задачи могут мигрировать между узлами
- локальная соседняя коммуникация существует
- оценка ресурсов доступна

Дополнительные механизмы требуются для:

- приоритетов задач
 - управления зависимостями
 - рабочих нагрузок с чувствительностью к задержкам
-

19. Будущая работа

Дальнейшее развитие может включать:

- много-ресурсную оптимизацию
- иерархические слои роя
- адаптивные параметры диффузии
- фреймворки симуляции

- экспериментальная валидация на кластерных системах
-

20. Заключение

Данная работа представляет распределённую рой-вычислительную архитектуру на основе динамики поля и потока задач–ресурсов.

Система заменяет централизованное планирование локальными взаимодействиями, которые коллективно балансируют нагрузки через всю сеть.

Модель математически обоснована графовой диффузионной динамикой и реализуема на реальной распределённой инфраструктуре.

Этот подход предлагает практический путь к децентрализованным вычислительным системам, способным координировать сотни или тысячи узлов без централизованного контроля.